INFORME TECNICO

Arquitectura SPA y modulo Clientes

PharmaSoft | Lenguaje de Programacion II | Sesion 7 - Actividad autonoma

B1. Estructura del proyecto

La organizacion por responsabilidades mantiene utilidades transversales separadas de cada dominio. El feature Clientes incorpora sus modelos, servicio, paginas y rutas; no absorbe el layout ni las reglas compartidas.

```
src/app/
|-- app.routes.ts
|-- core/  config/menu.ts; models/pagina-response.ts; utils/http-error.ts
|-- features/
|   |-- inicio/inicio.ts
|   |-- categorias/ models/; services/; pages/categoria-list/; pages/categoria-form/; categorias.routes.ts
|   `-- clientes/ models/; services/; pages/cliente-list/; pages/cliente-form/; clientes.routes.ts
|-- layout/ header/; main-layout/; sidebar/
`-- shared/pages/no-encontrado/
```

core concentra menu, tipos genericos y errores HTTP; **features** agrupa cada capacidad de negocio; **layout** contiene el cascaron persistente; **shared** aloja la pagina reutilizable de ruta inexistente. En este frontend no existe una carpeta separada llamada shared/components: sus componentes compartidos de navegacion viven en layout.

B2. Mapa de rutas

URL	Componente	Padre	Carga	Titulo
/	Redireccion	-	redirectTo	/inicio
/inicio	Inicio	MainLayout	loadComponent	Inicio - PharmaSoft
/categorias	CategoriaList	MainLayout	loadChildren + loadComponent	Categorias - PharmaSoft
/categorias/nuevo	CategoriaForm	MainLayout	loadChildren + loadComponent	Nueva categoria - PharmaSoft
/categorias/:id/editar	CategoriaForm	MainLayout	loadChildren + loadComponent	Editar categoria - PharmaSoft
/clientes	ClienteList	MainLayout	loadChildren + loadComponent	Clientes - PharmaSoft
/clientes/nuevo	ClienteForm	MainLayout	loadChildren + loadComponent	Nuevo cliente - PharmaSoft
/clientes/:id/editar	ClienteForm	MainLayout	loadChildren + loadComponent	Editar cliente - PharmaSoft
/**	NoEncontrado	Ninguno	loadComponent	Pagina no encontrada - PharmaSoft

Rutas y titulos: **app.routes.ts:4-29**; ramas de modulo: **categorias.routes.ts:3-7** y **clientes.routes.ts:3-7**.

B3. Responsabilidades y limites

La tabla distingue orquestacion de interfaz, acceso HTTP y composicion del shell. Categoria y Cliente siguen el mismo patron por feature.

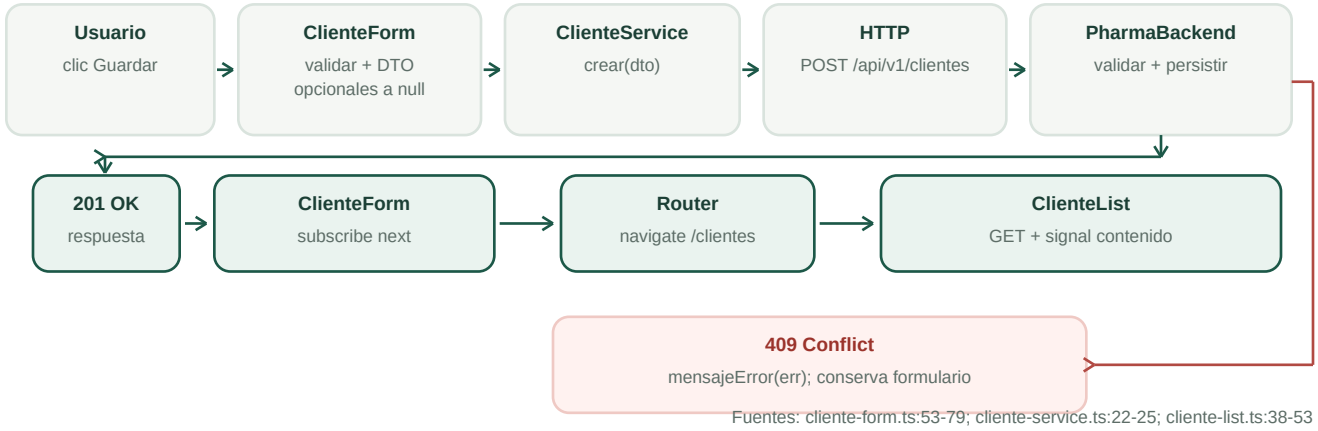
Pieza	Tipo	Responsabilidad	Que no hace
CategoriaList	Componente	Carga y presenta categorias; filtra, refleja carga/error/vacio y ofrece acciones.	No crea peticiones HTTP ni define reglas de persistencia.
CategoriaForm	Componente	Valida y captura datos para alta o edicion.	No administra la tabla ni construye la URL de API.
CategoriaService	Servicio	Centraliza operaciones HTTP tipadas del recurso categorias.	No muestra mensajes ni controla navegacion.
ClienteList	Componente	Mantiene pagina, tamanio, orden, filtro y estados; invoca el servicio y actualiza la tabla.	No usa HttpClient ni valida el formulario de alta/edicion.
ClienteForm	Componente	Precarga para editar; valida, normaliza opcionales, guarda y navega; muestra errores API.	No lista ni pagina clientes ni conoce detalles del repositorio.
ClienteService	Servicio	Cinco operaciones HTTP sobre /clientes con parametros y tipos de pagina.	No presenta UI, conserva estado visual ni navega.
MainLayout	Componente layout	Compone Header, Sidebar y RouterOutlet; mantiene el estado colapsado.	No define CRUD ni selecciona registros de negocio.
Header	Componente layout	Muestra marca, accesos superiores y emite la accion de colapsar.	No modifica el menu ni carga datos de modulos.
Sidebar	Componente layout	Renderiza MENU y marca el enlace activo segun la ruta.	No implementa rutas, autorizacion ni reglas de clientes.

Referencias: **ClienteList** cliente-list.ts:14-34, 38-53; **ClienteForm** cliente-form.ts:15-31, 53-79; **ClienteService** cliente-service.ts:8-25. Shell: main-layout.html:1-4, main-layout.ts:12-17, header.ts:10-12, sidebar.ts:11-13. Las piezas de Categorias siguen el mismo patron en features/categorias/pages y services.

Categorias: categoria-list.ts:14-43; categoria-form.ts:15-53; categoria-service.ts:7-17.

B4. Flujo: registrar un cliente

El formulario es el punto de entrada. Su submit valida en el navegador, prepara el DTO y delega la operacion al servicio. La tabla solo se refresca tras el 201 y la navegacion de retorno.



Fuentes: cliente-form.ts:53-79; cliente-service.ts:22-25; cliente-list.ts:38-53

Rama correcta: ClienteForm.guardar() bloquea entradas invalidas, transforma telefono/direccion vacios en null y llama crear(); el servicio emite el POST. Con 201, el callback navega a /clientes; ClienteList hace GET y reemplaza el signal clientes con respuesta.contenido. **Rama de conflicto:** ante 409, el callback usa mensajeError(), mantiene el formulario y detiene el indicador de carga para que se corrijan los datos.

Evidencia de codigo: **cliente-form.ts:53-79**, **cliente-service.ts:22-25**, **cliente-list.ts:38-53**; la funcion compartida de mensaje: **core/utils/http-error.ts:4-12**.

B5. Preguntas de revision

Respuestas basadas en el codigo de esta copia del repositorio.

1. Archivos existentes modificados.

En el **frontend**, integrar el feature requiere modificar dos archivos existentes: **app.routes.ts:19-22** agrega la ruta lazy de Clientes y **core/config/menu.ts:7-11** agrega su enlace. El resto se organiza en la carpeta nueva features/clientes. Esto muestra que la navegacion permite extender el producto con acoplamiento bajo. Nota de alcance: esta copia del backend no exponia el contrato paginado y baja logica descrito por la guia; para ejecutar A7 tambien se ajustaron ClienteController, ClienteService y ClienteServiceImpl y se agrego PaginaResponseDTO.

2. Por que ClienteList no usa HttpClient directamente.

La vista delega a **ClienteService** y depende de tipos/modelos, no de detalles de transporte (**cliente-list.ts:4-6, 15, 41**). El servicio concentra la URL basada en environment.apiUrl y las cinco operaciones (**cliente-service.ts:8-25**). Si cambia la ruta del recurso o la construccion de parametros, se actualiza el servicio (y solo la configuracion de entorno si cambia el host), sin reescribir el componente.

3. Ventaja de loadChildren.

loadChildren conserva el limite de cada feature y permite que el router cargue su configuracion bajo demanda, en vez de importar sus componentes directamente en la tabla raiz (**app.routes.ts:15-22**). En la compilacion verificada, Angular genero el chunk lazy **chunk-DFpgsSCs.js | cliente-list**, **chunk-BpDzsS5F.js | cliente-form** y **chunk-Dw5VmBm9.js | clientes-routes**. La salida mostro “Application bundle generation complete”.

4. Persistencia del encabezado y sidebar.

Ambos pertenecen a **MainLayout**, declarado como componente de la ruta padre vacia, y el template deja un unico **router-outlet** para las rutas hijas (**app.routes.ts:5-8**; **main-layout.html:1-4**). Al navegar entre categorias y clientes, el router reemplaza el componente hijo del outlet y conserva el padre; por eso el shell no se desmonta.

B5. Preguntas de revision (continuacion)

5. Donde aplicar visibilidad por permisos.

El punto central para filtrar los enlaces del sidebar es **core/config/menu.ts**, consumido por Sidebar (**sidebar.ts:1-13** y **menu.ts:7-11**). Si la visibilidad depende del usuario, MENU puede componerse o filtrarse desde esa configuracion/servicio de permisos, manteniendo una sola fuente de navegacion. La proteccion de una ruta y la autorizacion real de la API deben validarse aparte: ocultar el enlace solo resuelve presentacion.

6. Duplicacion y posible reutilizacion.

Los dos features repiten el ciclo lista/formulario/servicio, estados de carga y error, botones de paginacion, confirmaciones y presentacion de formularios. Conviene extraer controles visuales compartidos (campo con error, estado vacio, paginador) y, si aparecen mas recursos, una tabla generica parametrizada por columnas y acciones. Las reglas de Categoria y Cliente, DTO y servicios permanecen especificos; la abstraccion se justifica solo para comportamiento realmente igual.

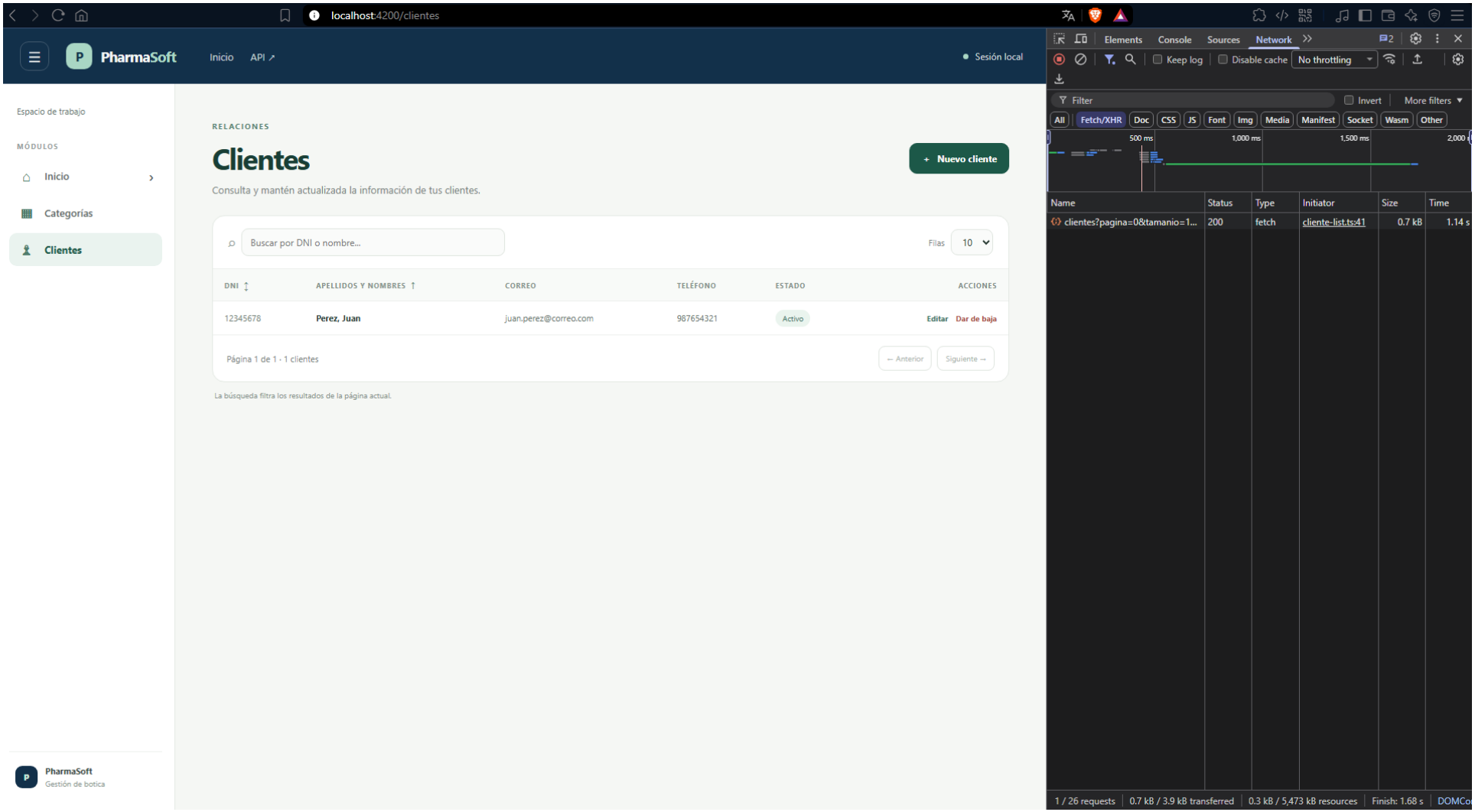
7. Alcance de la busqueda.

El computed filtra **this.clientes()**, que se asigna desde **respuesta.contenido** (**cliente-list.ts:27-34, 41-48**); ese arreglo representa solo la pagina solicitada. Para buscar globalmente, el backend debe aceptar un termino opcional y combinar el filtro con paginacion/orden, calculando totalElementos sobre los resultados filtrados. ClienteService debe agregar ese parametro a HttpParams y reiniciar pagina a cero cuando cambia la consulta. El repositorio debe buscar en DNI, nombres y apellidos sin distinguir mayusculas, y devolver la pagina filtrada.

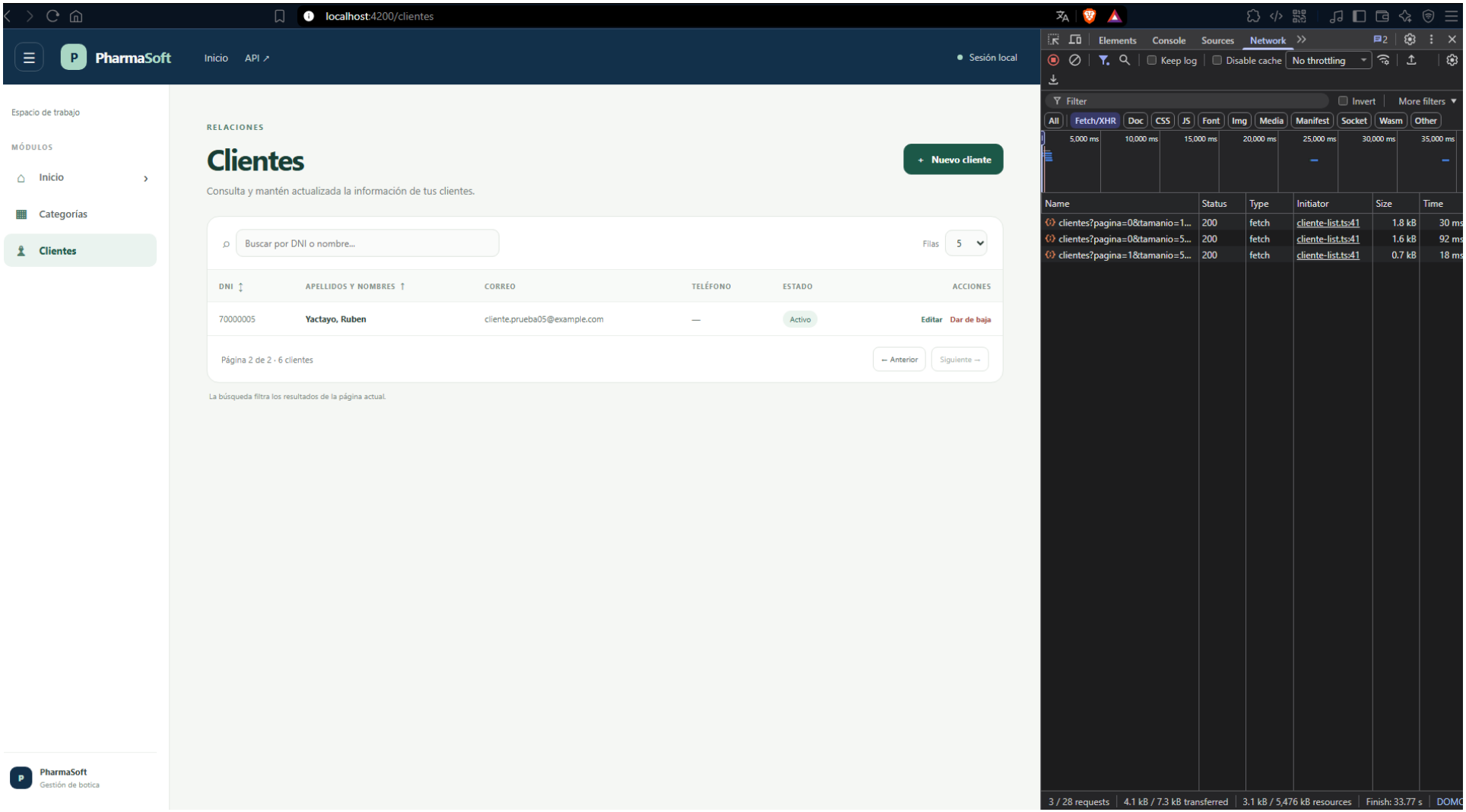
Verificacion de implementacion: el build Angular se completo y produjo chunks independientes para ClienteList, ClienteForm y clientes-routes. A7 se documento en el anexo, con la pestaña Network visible en cada captura.

Q6 - Codigo comparado: categoria-list.ts:14-43; categoria-form.ts:15-53; categoria-service.ts:7-17; cliente-list.ts:14-82; cliente-form.ts:15-94; cliente-service.ts:8-25.

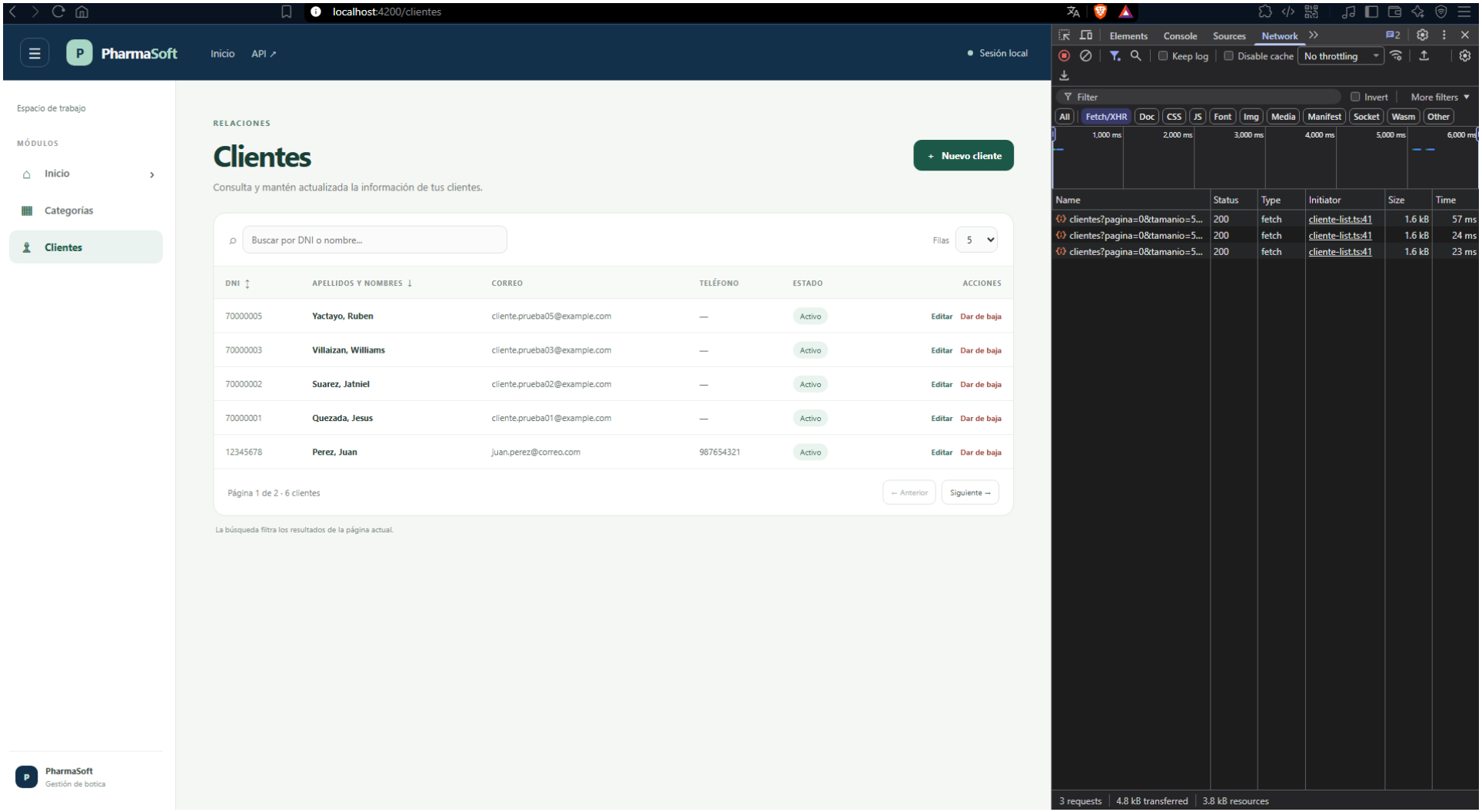
A7.1 - Navegacion desde el sidebar; listado inicial y GET 200.



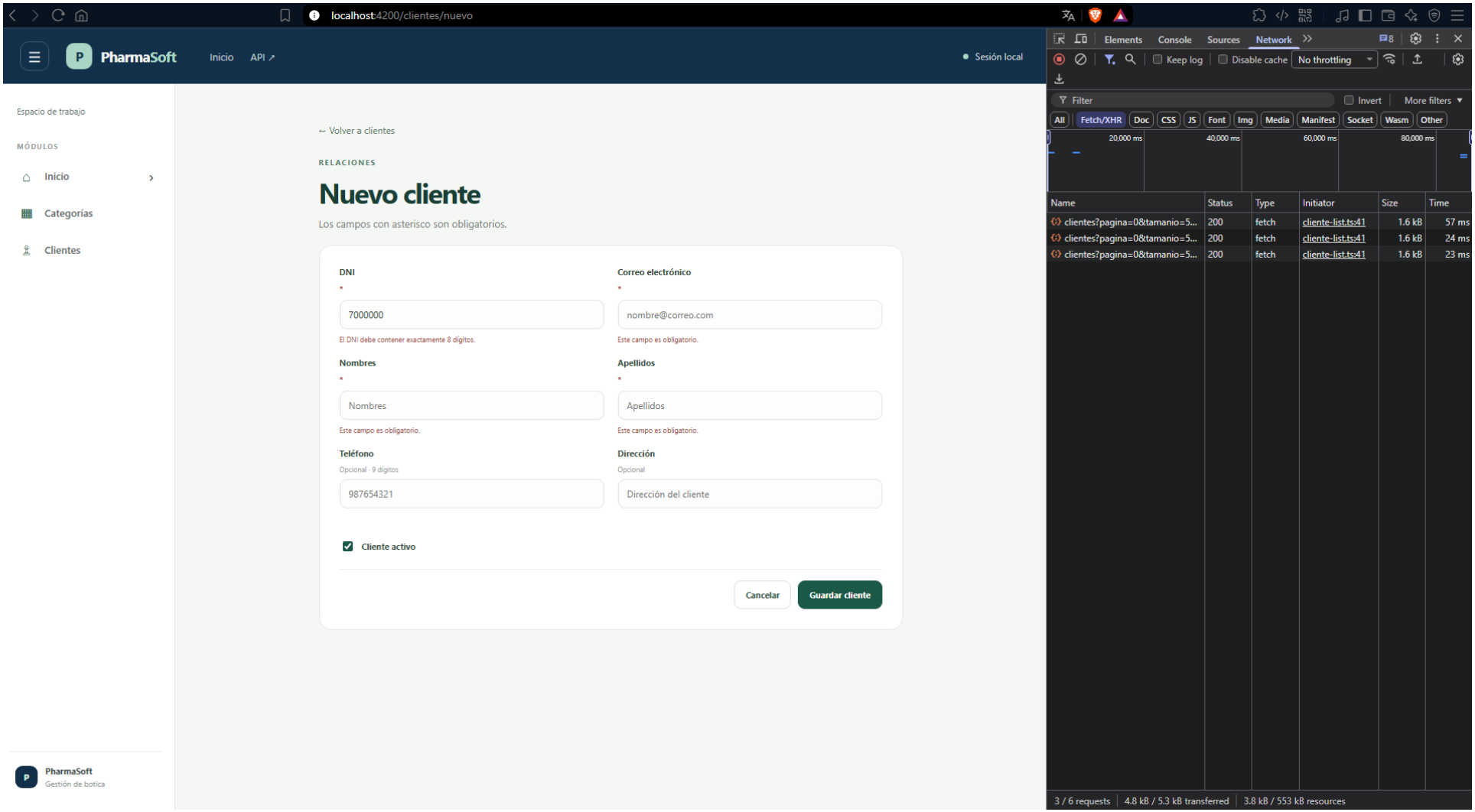
A7.2 - Tamano de pagina 5 y navegacion a pagina 2.



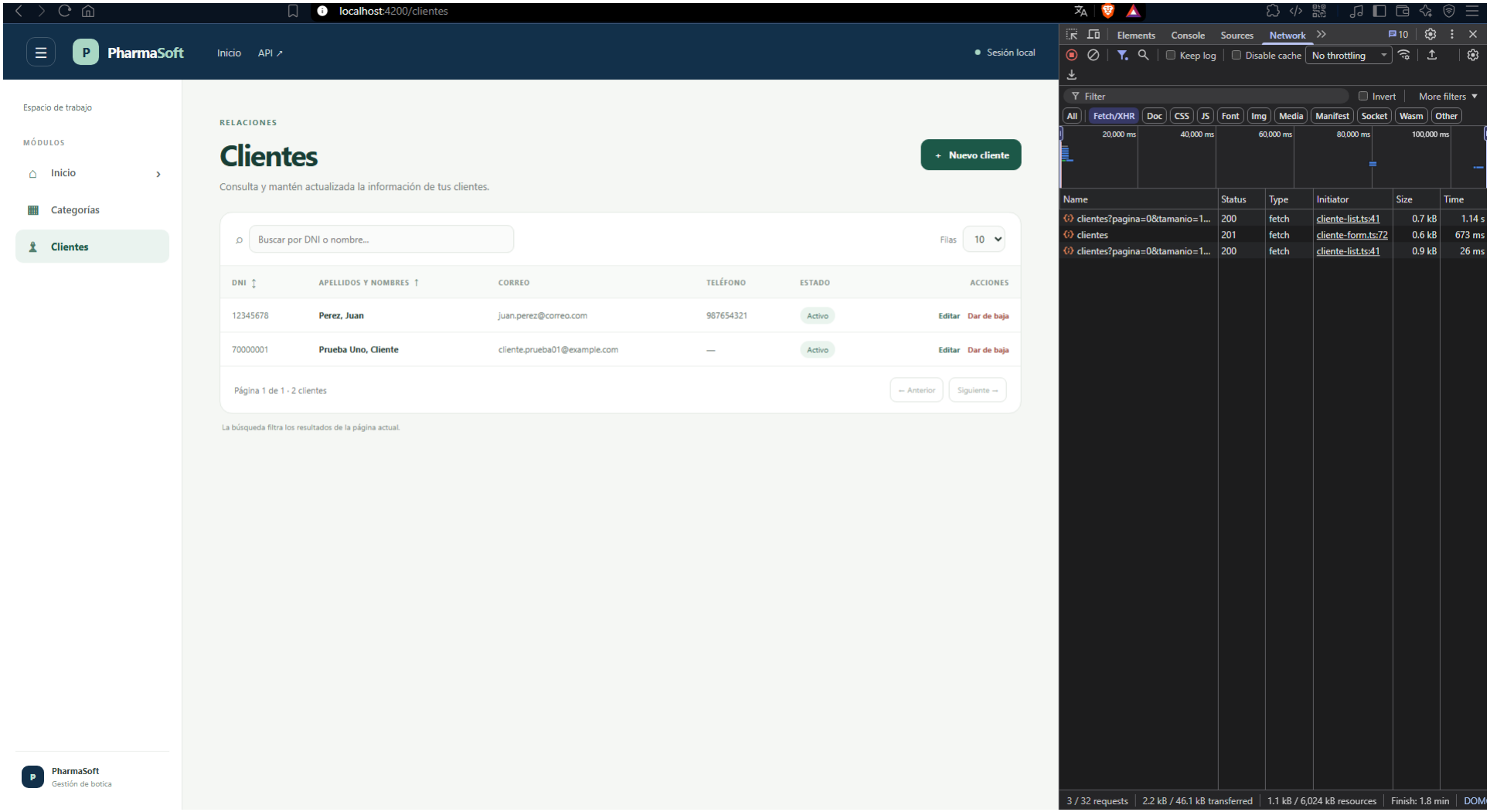
A7.3 - Orden por apellidos alternado; lista en orden descendente.



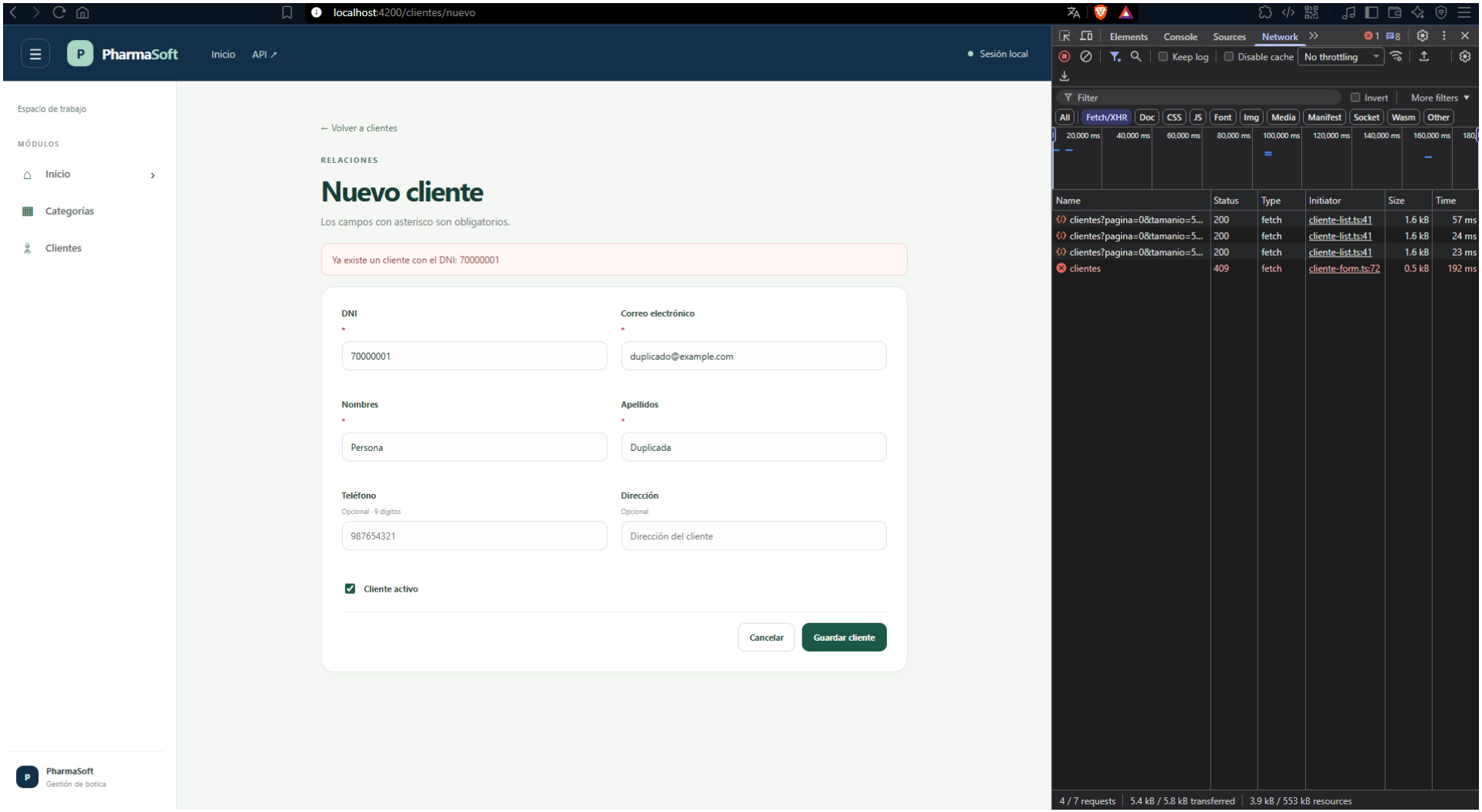
A7.4 - Validacion de DNI de siete digitos sin POST.



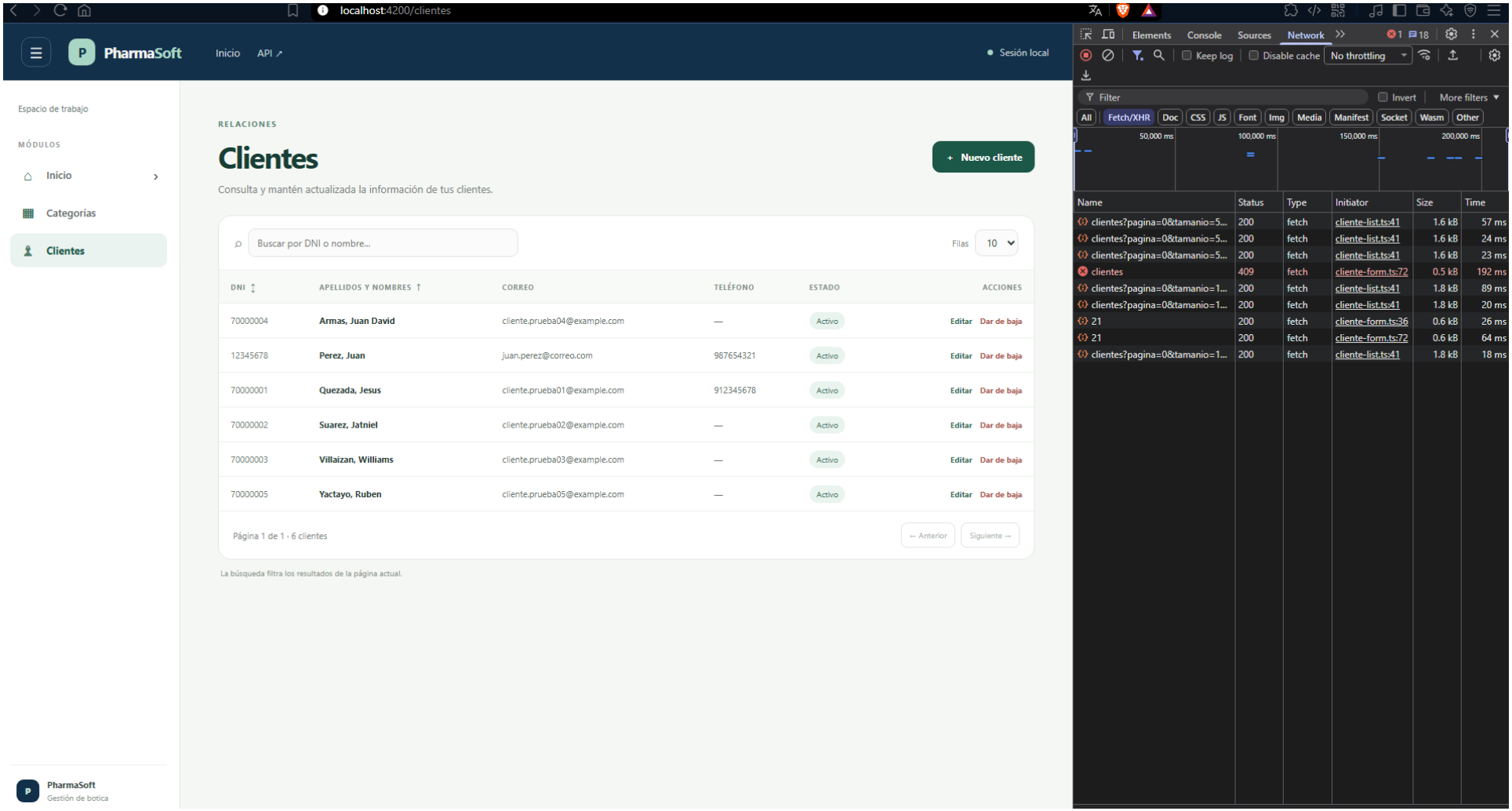
A7.5 - Alta valida sin telefono; POST 201 y telefono mostrado como raya.



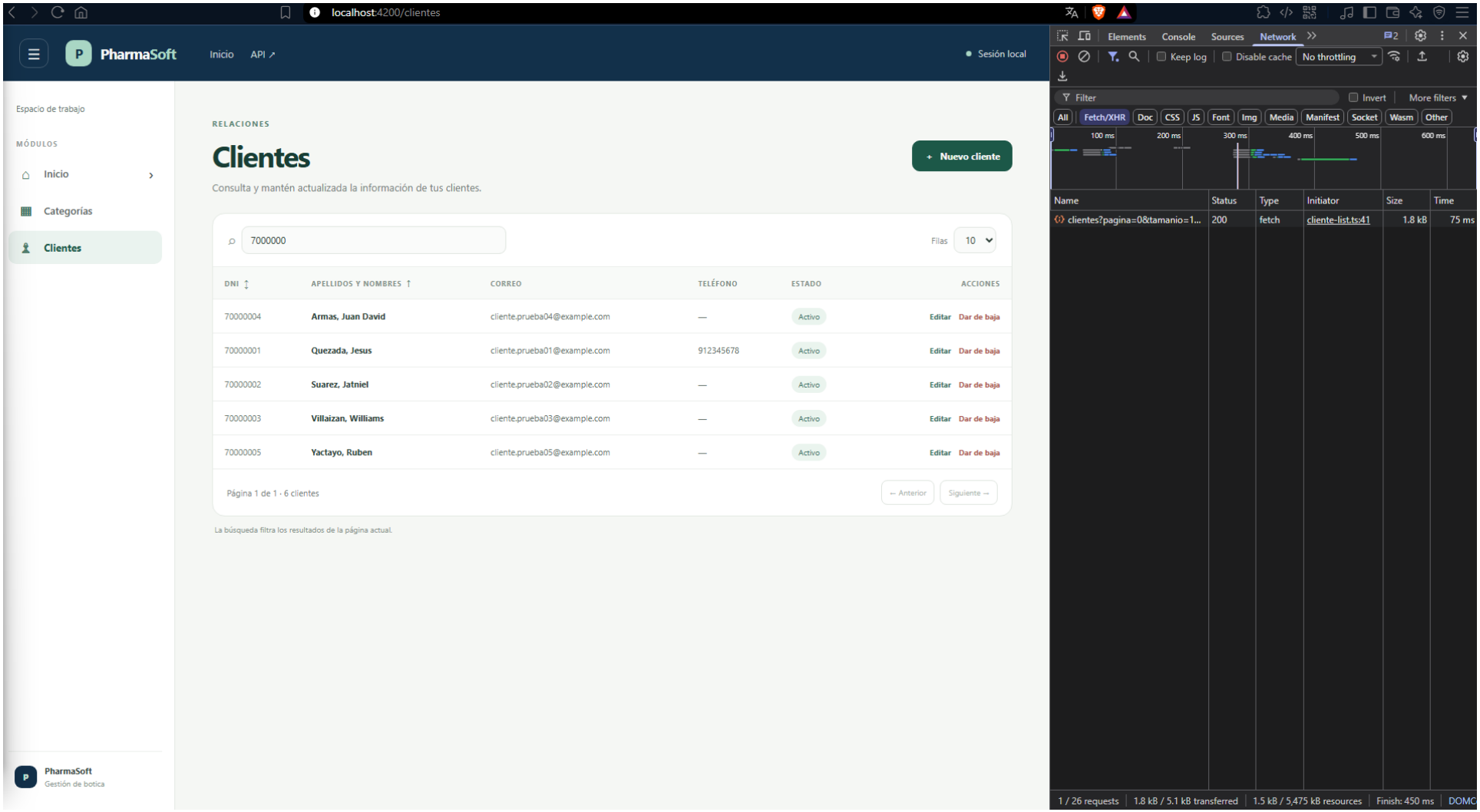
A7.6 - DNI duplicado; mensaje del backend y POST 409.



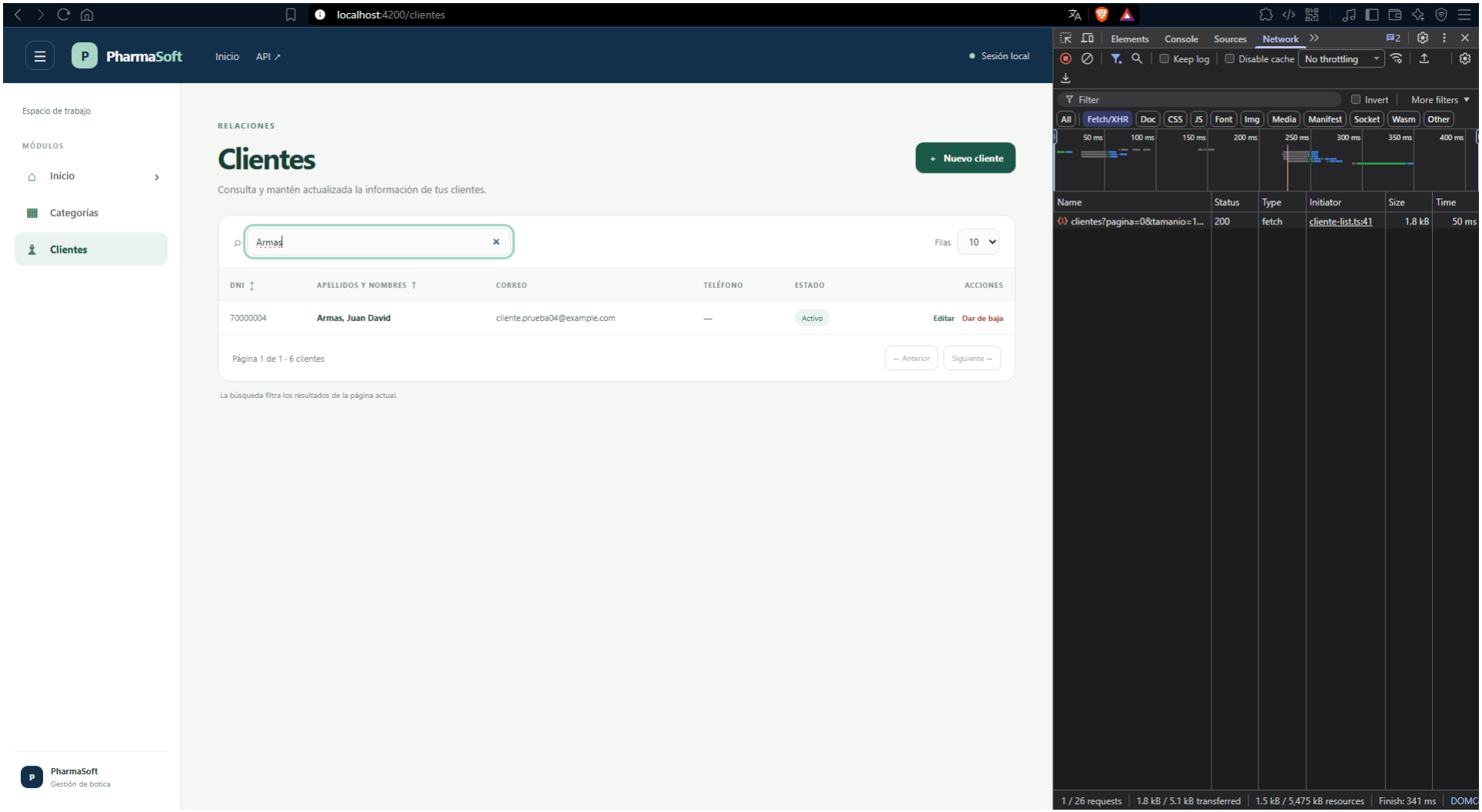
A7.7 - Edicion de telefono; fila actualizada y PUT 200.



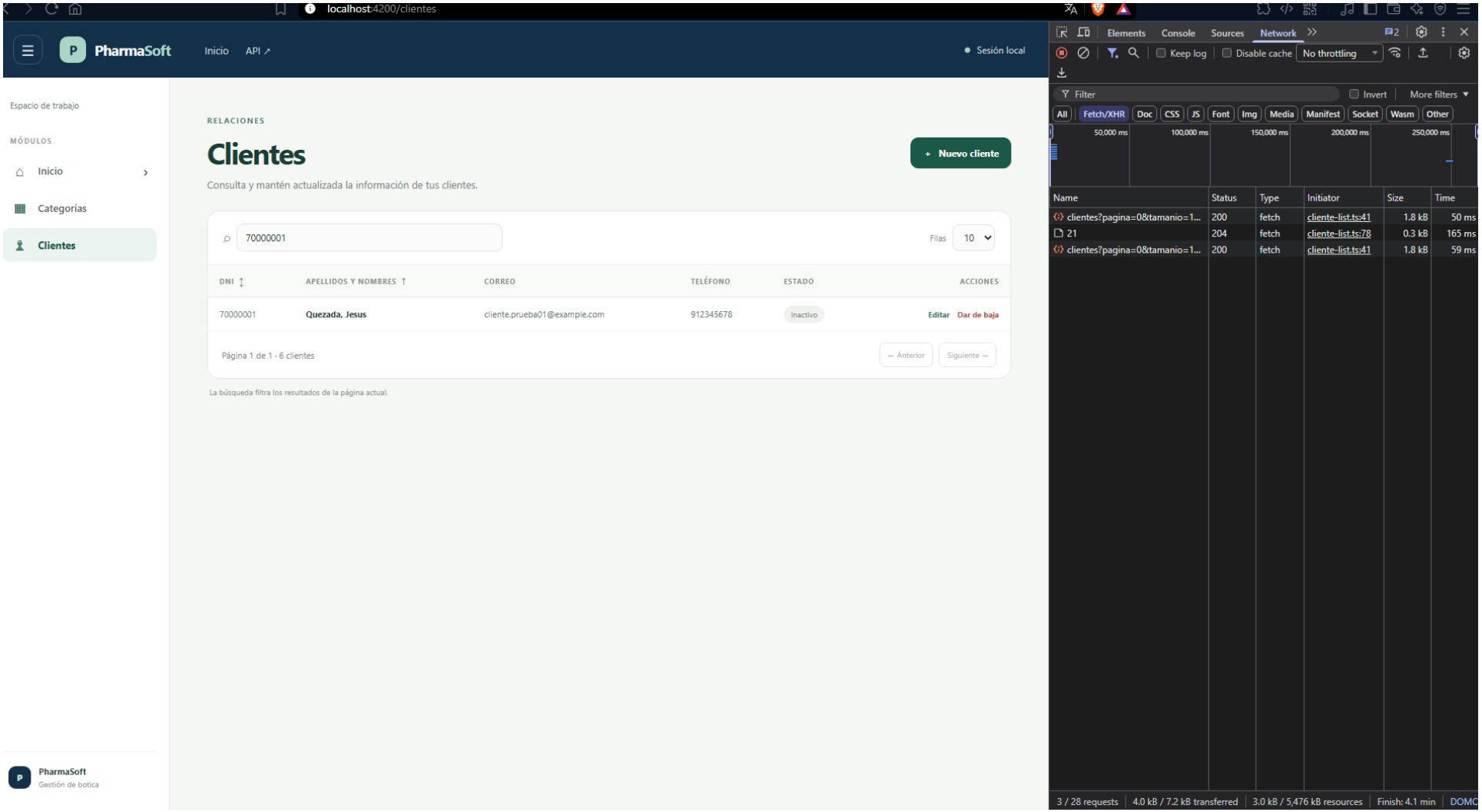
A7.8a - Busqueda por fragmento del DNI dentro de la pagina actual.



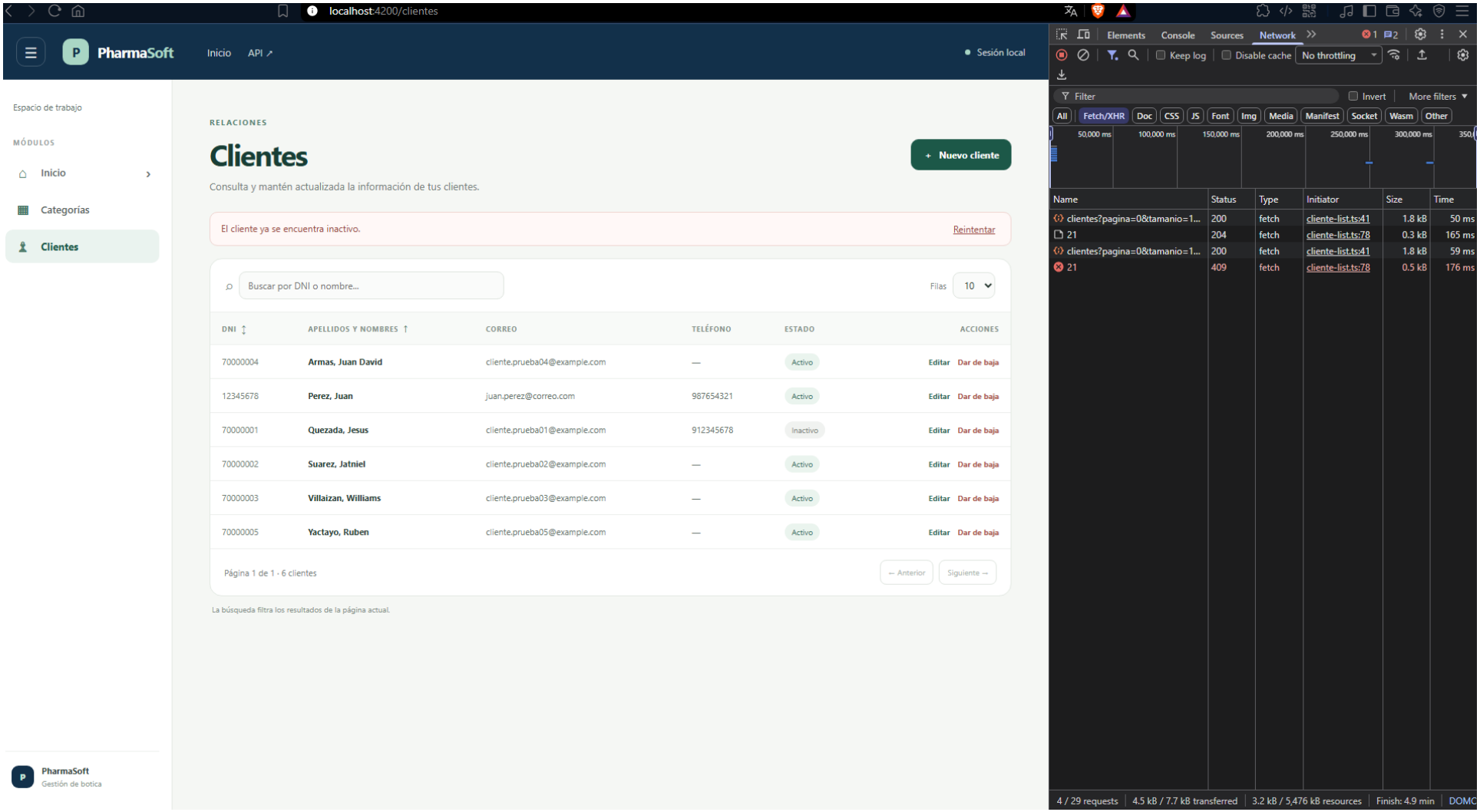
A7.8b - Busqueda por apellido; resultado Armas sin peticion nueva por el filtro.



A7.9 - Baja logica confirmada; cliente Inactivo y DELETE 204.



A7.10 - Segundo intento de baja; mensaje y DELETE 409.



A7.11 - Regreso a Categorias; catalogo cargado y GET 200.

